UNIVERSIDAD POLITECNICA DEL BICENTENARIO



REPORTE DE PRACTICAS

Práctica 1: Control de dispositivos básicos

Presenta:

Alejandro Almanza Perez

Vanessa Guadalupe Soria Zavala

Adán Rodriguez Ortiz

America Aranzazu Rodriguez Perez

Matricula:

24033177

24030215

24033563

2403

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto a 11 de septiembre de 2026

INDICE

Introducción ... 3
Marco teórico ... 3
Control ON-OFF de nivel. ... 8
PRÁCTICA 2 Sensor ultrasónico, bomba de diafragma. ... 8
Objetivo ... 8
Material requerido ... 8
Asignación de pines ... 8
Desarrollo ... 9
Diseño del circuito ... 9
Conexiones ... 11
Programación ... 11
Funcionamiento ... 22
Resultados ... 23
Observaciones y conclusiones ... 24
Referencias ... 26

Indice de Figuras

Figura 1. Circuito: Control ON-OFF de nivel. ... 10
Figura 2. Circuito — Lectura de nivel con parámetros configurables (sin bombas) . 13
Figura 3. Circuito físico con bombas sin activar ... 17
Figura 4. Circuito físico con bombas en funcionamiento. ... 17
Figura 5. Circuito físico funcionando ... 23
Figura 6. Circuito físico funcionando vista completa ... 23

Introducción

El control de nivel de líquidos es una de las tareas más comunes en la automatización de procesos. En tinacos, cisternas y tanques industriales es necesario mantener el volumen de un fluido dentro de un rango seguro sin vigilancia constante. Una de las estrategias más sencillas es el control ON-OFF, en el que una bomba se enciende o se apaga según el nivel medido.

En esta práctica se desarrolló un sistema de control ON-OFF de nivel con una tarjeta ESP32. Un sensor ultrasónico HC-SR04 mide la distancia a la superficie del líquido y una pantalla LCD I2C muestra el nivel y los parámetros. Tres botones ajustan los umbrales de nivel bajo (LL) y nivel alto (HL), y dos relevadores accionan las bombas de llenado y drenado. El sistema se construyó en tres etapas: monitoreo del nivel, control con bombas y una versión final más estable. En el proceso se presentaron fallas de conexión que se documentan junto con su solución.

El reporte incluye el marco teórico, el desarrollo de la práctica (objetivo, material, pines, diseño, conexiones, programación y funcionamiento), los resultados y las observaciones y conclusiones.

Marco teórico

Los sistemas de automatización industrial requieren dispositivos capaces de procesar información y tomar decisiones a partir de distintas condiciones de entrada. Para comprender el funcionamiento de la práctica desarrollada, es necesario abordar primero los conceptos de sistema embebido, microcontrolador, señales digitales, niveles lógicos y operaciones lógicas básicas.

Sistemas embebidos

Un sistema embebido es un sistema de cómputo diseñado para realizar una función específica, cuyo procesamiento central recae en un microcontrolador o microprocesador integrado junto con memoria e interfaces de entrada/salida en un mismo circuito. A diferencia de una computadora de propósito general, un sistema embebido se dedica a una tarea concreta, lo cual le permite ejecutarla con mayor precisión, rapidez y menor consumo de recursos.

Estos sistemas combinan hardware y software especializados: el hardware aporta los componentes físicos (sensores, actuadores, memoria e interfaces de comunicación), mientras que el software embebido gestiona su funcionamiento, procesa datos y controla la interacción con el usuario o con otros dispositivos. Entre sus principales ventajas se encuentran el control preciso de procesos, la reducción

de costos operativos, la posibilidad de un diseño modular adaptable a distintas necesidades y la capacidad de monitoreo en tiempo real, características que los hacen ideales para aplicaciones de control y automatización industrial.

El microcontrolador ESP32

El ESP32 es un microcontrolador de bajo costo desarrollado por Espressif Systems, que integra conectividad Wi-Fi y Bluetooth de doble modo. Cuenta con un procesador Xtensa de arquitectura de 32 bits (disponible en versiones de uno o dos núcleos) capaz de operar hasta 240 MHz, además de memoria SRAM y ROM integradas.

Entre sus características más relevantes para el desarrollo de proyectos de control se encuentran:

- Hasta 34 pines GPIO (General Purpose Input/Output) configurables como entradas o salidas digitales.
- Dos convertidores analógico-digitales (ADC) de 12 bits, con hasta 18 canales.
- Dos convertidores digital-analógicos (DAC) de 8 bits.
- Múltiples interfaces de comunicación: SPI, I2C, UART e I2S.
- Módulo de PWM con hasta 16 canales, útil para el control de motores y LEDs.

Gracias a la multiplexación de pines, la mayoría de las funciones del ESP32 pueden reasignarse a diferentes GPIO según las necesidades del proyecto. No obstante, existen pines de propósito especial —como los pines de arranque (strapping) o los de entrada únicamente (GPIO 34 a 39)— que deben evitarse o utilizarse con precaución, ya que intervienen en la configuración del microcontrolador durante el encendido.

Entradas y salidas digitales

Los microcontroladores interpretan el mundo físico a través de sus terminales de entrada y salida. Una entrada permite que el microcontrolador reciba una señal de voltaje y, a partir de ella, determine si un evento ha ocurrido (por ejemplo, si un botón fue presionado). Una salida, en cambio, es un terminal mediante el cual el microcontrolador entrega una señal de voltaje hacia un componente externo, como un LED.

Cuando estas señales solo pueden tomar dos valores discretos, se les denomina señales digitales, en contraposición a las señales analógicas, cuyo voltaje varía de forma continua en función de la magnitud que representan. En el caso de una entrada digital, el microcontrolador únicamente distingue dos estados: HIGH (nivel lógico alto) y LOW (nivel lógico bajo), sin importar variaciones intermedias de voltaje.

Niveles lógicos

Un nivel lógico es la tensión específica o el estado en el que puede encontrarse una señal digital. En electrónica digital se manejan dos estados: encendido (ON) y apagado (OFF), representados en binario como 1 y 0 respectivamente. El estándar más utilizado para definir estos niveles es la lógica TTL (Transistor-Transistor Logic), la cual establece rangos de voltaje aceptables tanto para la entrada como para la salida de un dispositivo digital.

Para un sistema TTL estándar de 5 V se definen los siguientes umbrales:

- **VIH** (tensión mínima de entrada para un nivel alto): aproximadamente 2 V.
- **VIL** (tensión máxima de entrada para un nivel bajo): aproximadamente 0.8 V.
- **VOH** (tensión mínima de salida para un nivel alto): aproximadamente 2.7 V.
- **VOL** (tensión máxima de salida para un nivel bajo): aproximadamente 0.5 V.

La diferencia entre los rangos tolerables de entrada y salida se conoce como margen de ruido, y representa la cantidad de tensión indeseada que un sistema puede tolerar sin interpretar erróneamente una señal. Este concepto es relevante al diseñar circuitos digitales, ya que garantiza que los niveles HIGH y LOW se reconozcan de forma confiable incluso ante pequeñas variaciones de voltaje.

Operación lógica AND

La operación AND requiere que todas las condiciones de entrada sean verdaderas para activar la salida.

Para dos entradas A y B: $\boldsymbol{Y = A * B}$ $\boldsymbol{(1)}$

Su tabla de verdad es:

Tabla 1. Tabla de verdad de la función AND

Entrada A	Entrada B	Salida Y
0	0	0
0	1	0
1	0	0
1	1	1

Operación lógica OR

La operación OR, a diferencia de la AND, activa la salida cuando al menos una de las entradas es verdadera.

Para dos entradas A y B: $\boldsymbol{Y = A + B}$ $\boldsymbol{(2)}$

Su tabla de verdad es:

Tabla 2. Tabla de verdad de la función OR

Entrada A	Entrada B	Salida Y
0	0	0
0	1	1
1	0	1
1	1	1

Control eléctrico y su aplicación en la automatización

El control eléctrico consiste en el uso de dispositivos eléctricos y electrónicos para regular, monitorear y automatizar el funcionamiento de sistemas y máquinas. Puede clasificarse en control manual, cuando requiere la intervención directa de un operador, o control automático, cuando se apoya en sensores, relés y controladores programables (PLC) que ajustan el comportamiento del sistema sin intervención humana constante. Este principio es la base conceptual de la lógica digital aplicada en la presente práctica, donde las entradas (botones) y la salida (LED) simulan, a pequeña escala, el comportamiento de un sistema de control automatizado.

Control ON-OFF de nivel con histéresis

El control ON-OFF, también conocido como de dos posiciones o todo-nada, solo puede encender o apagar el actuador según si la variable medida está por encima o por debajo de la referencia. [10] Si la salida cambia con cada pequeña variación de la variable, se acorta la vida del relevador de salida y pueden afectarse los dispositivos conectados, por lo que se crea una banda de histéresis entre los puntos de encendido y apagado. [9]

En esta práctica, los umbrales LL y HL forman esa banda. El nivel de llenado actual (AL) se obtiene como porcentaje a partir de las distancias de referencia del tanque vacío (d_{vacio}) y lleno ($d_{llenado}$):

$$AL = \frac{d_{vacio} - d}{d_{vacio} - d_{lleno}} * 100 \ (1)$$

Con ese porcentaje el sistema decide si llena el tanque, lo drena o espera.

Sensor ultrasónico HC-SR04

El HC-SR04 mide distancias sin contacto de 2 a 400 cm con una precisión de hasta 3 mm. El módulo integra un transmisor ultrasónico, un receptor y un circuito de control, y se conecta con cuatro pines: VCC, TRIG, ECHO y GND. [11] Para iniciar una medición se aplica un pulso de 10 µs al pin TRIG, y el sensor trabaja con ondas ultrasónicas de 40 kHz. [12] La distancia se calcula multiplicando el tiempo que ECHO permanece en alto por la velocidad del sonido (340 m/s) y dividiendo entre dos, porque la onda recorre el trayecto de ida y vuelta. [11]

$$d = \frac{t * v}{2} \quad (2)$$

Aplicado a un tanque, el sensor se coloca sobre la superficie del líquido y la distancia medida disminuye conforme el nivel sube.

Bus I2C y pantalla LCD

El ESP32 tiene dos interfaces I2C que pueden operar como maestro o esclavo. Por defecto usa GPIO 21 (SDA) y GPIO 22 (SCL), pero estos pines pueden configurarse por software. Las líneas SDA y SCL requieren resistencias pull-up. [13] Cada dispositivo del bus tiene su propia dirección, por lo que varios periféricos pueden compartir las mismas dos líneas. [13]

Una pantalla LCD I2C se compone de un display de caracteres basado en el controlador HD44780 y un adaptador I2C que convierte los datos del bus en la información paralela que el display necesita. Esto reduce la conexión a cuatro pines. [14] La pantalla se controla con la librería `LiquidCrystal_I2C`. En esta práctica se usaron pines I2C distintos a los predeterminados (GPIO 26 y 27), configurados con `Wire.begin()`.

Relevador

Un relevador funciona como un interruptor automático en el que una corriente pequeña controla una corriente mayor. Se usa para no conectar directamente un interruptor a un circuito de corriente alterna, lo que representaría un riesgo para las personas. El módulo cuenta con terminales NO (normalmente abierto) y NC (normalmente cerrado). [15] En una conexión normalmente abierta, al activarse el relevador el contacto COM se une con NO y la carga recibe energía. Así, el ESP32 solo maneja la señal de control, y las bombas, con su fuente externa, quedan en el circuito de potencia.

Control ON-OFF de nivel.

PRÁCTICA 2 Sensor ultrasónico, bomba de diafragma.

Objetivo

Implementar un sistema de monitoreo y control automático del nivel de líquido en un tanque, utilizando una tarjeta ESP32, un sensor ultrasónico HC-SR04 para medir la distancia a la superficie del líquido, una pantalla LCD I2C para visualizar los parámetros y tres botones para ajustar los umbrales de nivel bajo (LL) y nivel alto (HL). A partir de estos umbrales, el sistema activa mediante dos relevadores una bomba de diafragma de llenado y otra de drenado, operando de forma cíclica entre ambos niveles. Con ello se aplican los conceptos de sensores ultrasónicos, comunicación I2C, filtrado de mediciones, máquinas de estado y control de cargas mediante relevadores.

Material requerido

- 1 tarjeta ESP32.
- 1 sensor ultrasónico HC-SR04.
- 1 pantalla LCD 16x2 con módulo adaptador I2C.
- 3 botones pulsadores (SELECT, UP, DOWN).
- 2 módulos relevadores de 1 canal.
- 2 bombas de diafragma (una para llenado y otra para drenado).
- Fuente de alimentación externa para las bombas.
- Tanque o contenedor de pruebas.
- Mangueras de conexión.
- Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE.

Asignación de pines

Componente	Pin ESP32	Tipo	Función
Sensor HC-SR04 (TRIG)	GPIO 13	Salida digital	Emite el pulso ultrasónico
Sensor HC-SR04 (ECHO)	GPIO 14	Entrada digital	Recibe el eco; su duración se convierte en distancia
Botón SELECT	GPIO 18	Entrada digital (INPUT_PULLUP)	Presionado junto con UP/DOWN edita HL; sin él, edita LL

Botón UP	GPIO 32	Entrada digital (INPUT_PULLUP)	Incrementa en 5 % el parámetro seleccionado
Botón DOWN	GPIO 19	Entrada digital (INPUT_PULLUP)	Decrementa en 5 % el parámetro seleccionado
LCD I2C (SDA)	GPIO 26	I2C (datos)	Comunicación con la pantalla (dirección 0x27)
LCD I2C (SCL)	GPIO 27	I2C (reloj)	Comunicación con la pantalla
Relevador de llenado (IN)	GPIO 4	Salida digital (activo en bajo)	Activa la bomba de llenado
Relevador de drenado (IN)	GPIO 5	Salida digital (activo en bajo)	

Desarrollo

Diseño del circuito

El sistema se desarrolló en tres etapas, cada una con un objetivo distinto". Deja las Etapas 1 y 2 como las tienes y agrega lo siguiente.

Etapa 1 — Lectura de nivel configurable: en esta primera versión se implementó únicamente el sensor ultrasónico y la pantalla LCD, junto con los tres botones de configuración. El sensor HC-SR04 mide la distancia entre el sensor y la superficie del líquido; a partir de una distancia de referencia con el tanque vacío (20 cm) y otra con el tanque lleno (4 cm), el programa calcula el porcentaje de llenado actual (AL) y lo convierte a litros, considerando una capacidad de tanque predefinida de 2 litros. Los botones UP y DOWN permiten modificar los umbrales de nivel bajo (LL) y nivel alto (HL), mientras que SELECT alterna cuál de los dos parámetros se está editando. Esta etapa permitió validar que la lectura del sensor y la interfaz de usuario funcionaran correctamente antes de agregar el control de las bombas.

Etapa 2 — Control automático de bombas: sobre la base funcional de la primera etapa, se agregaron dos relevadores para controlar una bomba de diafragma (llenado) y una segunda bomba o electroválvula (drenado). El sistema implementa una máquina de estados con tres posiciones: `ESPERA`, `LLENANDO` y `DRENANDO`. Cuando el nivel actual (AL) cae por debajo del umbral bajo (LL), el sistema activa el relevador de llenado; cuando el nivel supera el umbral alto (HL), se activa el relevador de drenado. Se incorporó un margen de histéresis de 2% para evitar que las bombas se enciendan y apaguen de forma intermitente cuando el nivel se encuentra muy cerca de los umbrales. Adicionalmente, se añadió un modo de configuración del tanque (`CONFIG_TANQUE`), accesible manteniendo presionados simultáneamente SELECT y UP durante 2 segundos, donde es posible ajustar el radio y la altura útil del tanque para recalcular su capacidad real en litros. Estos parámetros, junto con

los umbrales LL y HL, se almacenan en memoria no volátil mediante la librería `Preferences`, de modo que se conservan aún después de reiniciar o desconectar el ESP32.

Etapa 3 — Versión final: a partir de las fallas de la etapa anterior, se rediseñaron conexiones y programa para obtener un sistema más estable. Los relevadores se reubicaron en los GPIO 4 (llenado) y GPIO 5 (drenado) y se configuraron con lógica activa en nivel bajo: el relevador se activa cuando el pin entrega LOW y permanece desactivado con HIGH. El tanque de pruebas tiene una distancia de referencia de 11.5 cm cuando está vacío y de 4 cm cuando está lleno (altura útil de 7.5 cm), con capacidad fija de 0.589 L, por lo que se eliminaron el modo de configuración del tanque y el almacenamiento en memoria no volátil.

En el software, el programa dejó de usar retardos bloqueantes y se temporizó con millis(): los botones se leen cada 150 ms (en pasos de 5 %) y el sensor se mide cada 100 ms. Para reducir lecturas erráticas del HC-SR04, cada medición se obtiene de cinco lecturas consecutivas: se descartan las fuera de rango (menores a 0.5 cm o mayores a 30 cm) y se toma la mediana. Además, cada 2 segundos se verifica que la LCD responda en la dirección 0x27; si no responde, se reinicia el bus I2C y la pantalla, lo que complementa la corrección física del mal contacto del adaptador. Por último, la lógica de control pasó a un modo cíclico (llamado “modo fuente” en el código): el sistema llena el tanque hasta alcanzar HL, cambia a drenar hasta alcanzar LL y vuelve a llenar, repitiendo el ciclo continuamente.

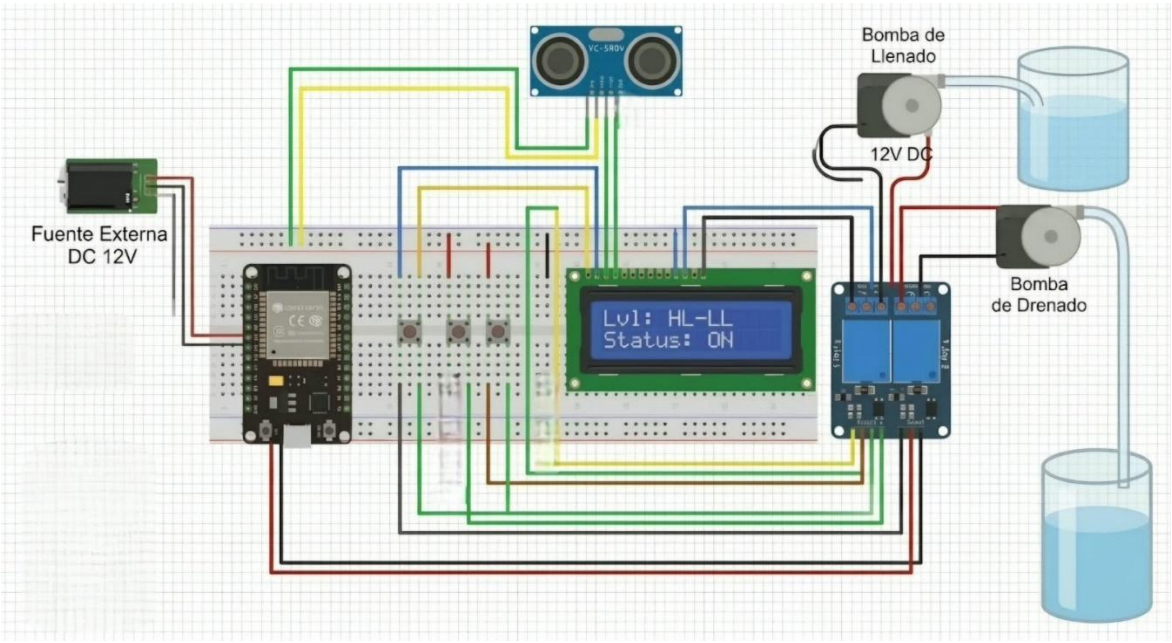


Figura 1. Circuito: Control ON-OFF de nivel.

Conexiones

- **GPIO 13 (TRIG) y GPIO 14 (ECHO) → Sensor HC-SR04:** emisión del pulso y recepción del eco; el sensor se alimenta a 5 V y GND.
- **GPIO 26 (SDA) y GPIO 27 (SCL) → Módulo I2C de la LCD:** datos y reloj de la pantalla.
- **GPIO 18 → Botón SELECT → GND:** selecciona el parámetro a editar.
- **GPIO 32 → Botón UP → GND:** incrementa el valor seleccionado.
- **GPIO 19 → Botón DOWN → GND:** decrementa el valor seleccionado.
- **GPIO 4 → Pin IN del relevador de llenado:** activa la bomba de llenado al recibir nivel bajo.
- **GPIO 5 → Pin IN del relevador de drenado:** activa la bomba de drenado al recibir nivel bajo.
- **VCC y GND de los relevadores → 5 V y GND del ESP32**, con GND común entre los circuitos.
- **Contactos de potencia de los relevadores (COM/NO) → Bombas y fuente externa:** circuito de potencia aislado del de control.

Programación

Versión 1 — Lectura de nivel con parámetros configurables (sin bombas):

```
#include "Wire.h"
#include "LiquidCrystal_I2C.h"

// Pines del sensor ultrasónico
const int TRIG_PIN = 13;
const int ECHO_PIN = 14;

// Pines de los botones
const int BTN_SELECT = 18;
const int BTN_UP     = 32;
const int BTN_DOWN   = 19;

// Parámetros del tanque
const float DISTANCIA_VACIO = 20.0;
const float DISTANCIA_LLENO = 4.0;
const float ALTURA_MAXIMA = DISTANCIA_VACIO - DISTANCIA_LLENO;
const float CAPACIDAD_LITROS = 2.0;

// Variables de estado
int LL = 20;
int HL = 80;
int AL = 0;
float litrosActuales = 0.0;

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(BTN_SELECT, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
```

```
  Wire.begin(26, 27);
  lcd.init();
  lcd.backlight();
}

void loop() {
  bool editandoHL = (digitalRead(BTN_SELECT) == LOW);

  if (digitalRead(BTN_UP) == LOW) {
    if (editandoHL) {
      HL++;
      if (HL > 100) HL = 100;
    } else {
      LL++;
      if (LL >= HL) LL = HL - 1;
    }
    delay(250);
  }

  if (digitalRead(BTN_DOWN) == LOW) {
    if (editandoHL) {
      HL--;
      if (HL <= LL) HL = LL + 1;
    } else {
      LL--;
      if (LL < 0) LL = 0;
    }
    delay(250);
  }

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duracion = pulseIn(ECHO_PIN, HIGH, 30000);
  float distancia = duracion * 0.0343 / 2;
  if (distancia < DISTANCIA_LLENO || distancia <= 0) distancia = DISTANCIA_LLENO;

  float nivelAguaCm = DISTANCIA_VACIO - distancia;
  if (nivelAguaCm < 0) nivelAguaCm = 0;
  if (nivelAguaCm > ALTURA_MAXIMA) nivelAguaCm = ALTURA_MAXIMA;

  AL = (int)((nivelAguaCm / ALTURA_MAXIMA) * 100.0);
  litrosActuales = (AL / 100.0) * CAPACIDAD_LITROS;

  char fila0[17];
  char fila1[17];
  snprintf(fila0, sizeof(fila0), "LL %-3d   AL %-3d ", LL, AL);
  snprintf(fila1, sizeof(fila1), "HL %-3d   L: %-4.1f", HL, litrosActuales);

  lcd.setCursor(0, 0);
  lcd.print(fila0);
  lcd.setCursor(0, 1);
  lcd.print(fila1);

  delay(100);
}
```

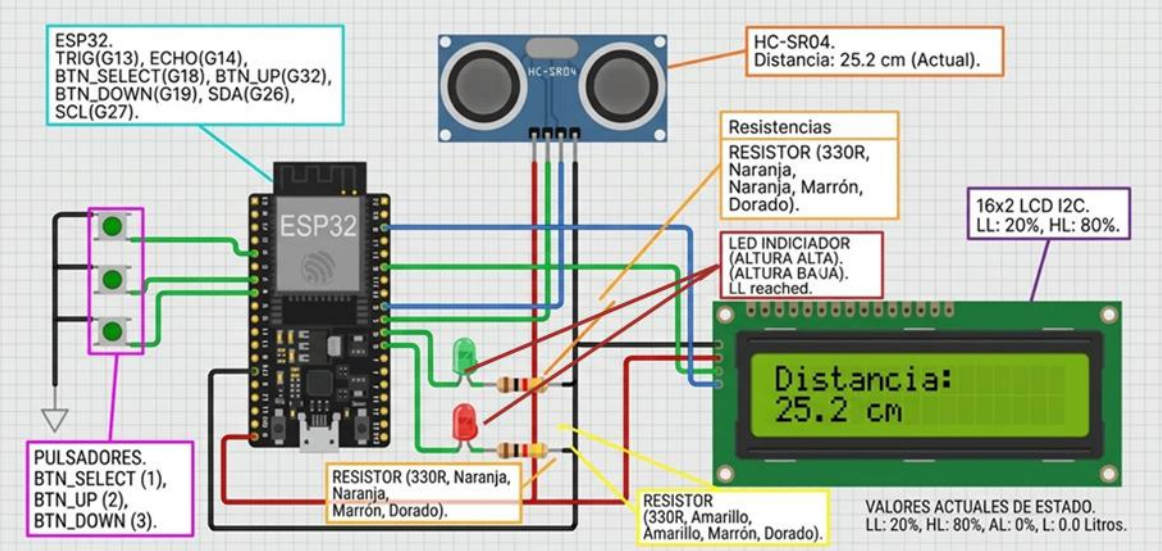


Figura 2. Circuito — Lectura de nivel con parámetros configurables (sin bombas)

Versión 2 — Control automático de bombas mediante relevadores:

```
#include "Wire.h"
#include "LiquidCrystal_I2C.h"
#include <Preferences.h>

// Pines del sensor ultrasónico
const int TRIG_PIN = 13;
const int ECHO_PIN = 14;

// Pines de los botones
const int BTN_SELECT = 18;
const int BTN_UP     = 32;
const int BTN_DOWN   = 19;

// Pines de los relés (dos bombas)
const int RELAY_LLENAR = 16;
const int RELAY_DRENAR = 17;

const int RELAY_ON  = HIGH;
const int RELAY_OFF = LOW;

// Parámetros del sensor
const float DISTANCIA_VACIO = 20.0;
const float DISTANCIA_LLENO = 4.0;
const float ALTURA_MAXIMA   = DISTANCIA_VACIO - DISTANCIA_LLENO;
const float MI_PI = 3.14159265;
const int MARGEN = 2;

// Variables configurables
float radioTanque = 6.3;
float alturaUtil  = 16.0;
int LL = 20;
int HL = 80;

int AL = 0;
float litrosActuales = 0.0;
float capacidadLitros = 0.0;

enum Estado { ESPERA, LLENANDO, DRENANDO };
```

```
Estado estadoActual = ESPERA;

enum Modo { NORMAL, CONFIG_TANQUE };
Modo modoActual = NORMAL;

unsigned long inicioCombo = 0;
bool comboActivo = false;

Preferences prefs;
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BTN_SELECT, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);

  pinMode(RELAY_LLENAR, OUTPUT);
  pinMode(RELAY_DRENAR, OUTPUT);
  digitalWrite(RELAY_LLENAR, RELAY_OFF);
  digitalWrite(RELAY_DRENAR, RELAY_OFF);

  Wire.begin(26, 27);
  lcd.init();
  lcd.backlight();

  prefs.begin("tanque", false);
  radioTanque = prefs.getFloat("radio", 6.3);
  alturaUtil  = prefs.getFloat("altura", 16.0);
  LL = prefs.getInt("LL", 20);
  HL = prefs.getInt("HL", 80);
  prefs.end();

  recalcularCapacidad();
}

void recalcularCapacidad() {
  capacidadLitros = (MI_PI * radioTanque * radioTanque * alturaUtil) / 1000.0;
}

void guardarEnMemoria() {
  prefs.begin("tanque", false);
  prefs.putFloat("radio", radioTanque);
  prefs.putFloat("altura", alturaUtil);
  prefs.putInt("LL", LL);
  prefs.putInt("HL", HL);
  prefs.end();
}

void actualizarMotores() {
  if (AL <= LL && estadoActual != LLENANDO) {
    estadoActual = LLENANDO;
  }
  else if (estadoActual == LLENANDO && AL >= (HL - MARGEN)) {
    estadoActual = ESPERA;
  }

  if (AL > HL && estadoActual != DRENANDO) {
    estadoActual = DRENANDO;
  }
  else if (estadoActual == DRENANDO && AL <= (LL + MARGEN)) {
    estadoActual = ESPERA;
  }

  switch (estadoActual) {
    case LLENANDO:
```

```
      digitalWrite(RELAY_LLENAR, RELAY_ON);
      digitalWrite(RELAY_DRENAR, RELAY_OFF);
      break;
    case DRENANDO:
      digitalWrite(RELAY_LLENAR, RELAY_OFF);
      digitalWrite(RELAY_DRENAR, RELAY_ON);
      break;
    case ESPERA:
    default:
      digitalWrite(RELAY_LLENAR, RELAY_OFF);
      digitalWrite(RELAY_DRENAR, RELAY_OFF);
      break;
  }
}

void loop() {
  bool select = (digitalRead(BTN_SELECT) == LOW);
  bool up     = (digitalRead(BTN_UP) == LOW);
  bool down   = (digitalRead(BTN_DOWN) == LOW);

  if (select && up) {
    if (!comboActivo) {
      comboActivo = true;
      inicioCombo = millis();
    } else if (millis() - inicioCombo > 2000) {
      modoActual = CONFIG_TANQUE;
      comboActivo = false;
      delay(500);
      return;
    }
  } else {
    comboActivo = false;
  }

  if (modoActual == NORMAL) {
    if (select) {
      if (up)   { HL++; if (HL > 100) HL = 100; delay(250); }
      if (down) { HL--; if (HL <= LL) HL = LL + 1; delay(250); }
    } else {
      if (up)   { LL++; if (LL >= HL) LL = HL - 1; delay(250); }
      if (down) { LL--; if (LL < 0) LL = 0; delay(250); }
    }
  }
  else if (modoActual == CONFIG_TANQUE) {
    if (up) {
      if (select) { alturaUtil += 0.5; if (alturaUtil > 100) alturaUtil = 100;
}
      else        { radioTanque += 0.1; if (radioTanque > 50) radioTanque = 50;
}
      recalcularCapacidad();
      delay(200);
    }
    if (down) {
      if (select) { alturaUtil -= 0.5; if (alturaUtil < 1) alturaUtil = 1; }
      else        { radioTanque -= 0.1; if (radioTanque < 0.5) radioTanque =
0.5; }
      recalcularCapacidad();
      delay(200);
    }
    if (select && !up && !down) {
      guardarEnMemoria();
      modoActual = NORMAL;
      delay(500);
    }

    char l0[17], l1[17];
    snprintf(l0, sizeof(l0), "CONFIG TANQUE   ");
    snprintf(l1, sizeof(l1), "R:%-4.1f H:%-4.1f", radioTanque, alturaUtil);
    lcd.setCursor(0, 0); lcd.print(l0);
```

```
    lcd.setCursor(0, 1); lcd.print(l1);
    return;
  }

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duracion = pulseIn(ECHO_PIN, HIGH, 30000);
  float distancia = duracion * 0.0343 / 2;
  if (distancia < DISTANCIA_LLENO || distancia <= 0) distancia =
DISTANCIA_LLENO;

  float nivelAguaCm = DISTANCIA_VACIO - distancia;
  if (nivelAguaCm < 0) nivelAguaCm = 0;
  if (nivelAguaCm > ALTURA_MAXIMA) nivelAguaCm = ALTURA_MAXIMA;

  AL = (int)((nivelAguaCm / ALTURA_MAXIMA) * 100.0);
  litrosActuales = (AL / 100.0) * capacidadLitros;

  actualizarMotores();

  char fila0[17];
  char fila1[17];
  const char* indicador = "  ";
  switch (estadoActual) {
    case LLENANDO: indicador = "^^"; break;
    case DRENANDO: indicador = "vv"; break;
    case ESPERA:   indicador = "--"; break;
  }

  snprintf(fila0, sizeof(fila0), "LL %-3d AL %-3d %s", LL, AL, indicador);
  snprintf(fila1, sizeof(fila1), "HL %-3d L: %-4.1f  ", HL, litrosActuales);

  lcd.setCursor(0, 0); lcd.print(fila0);
  lcd.setCursor(0, 1); lcd.print(fila1);

  delay(100);
}
```

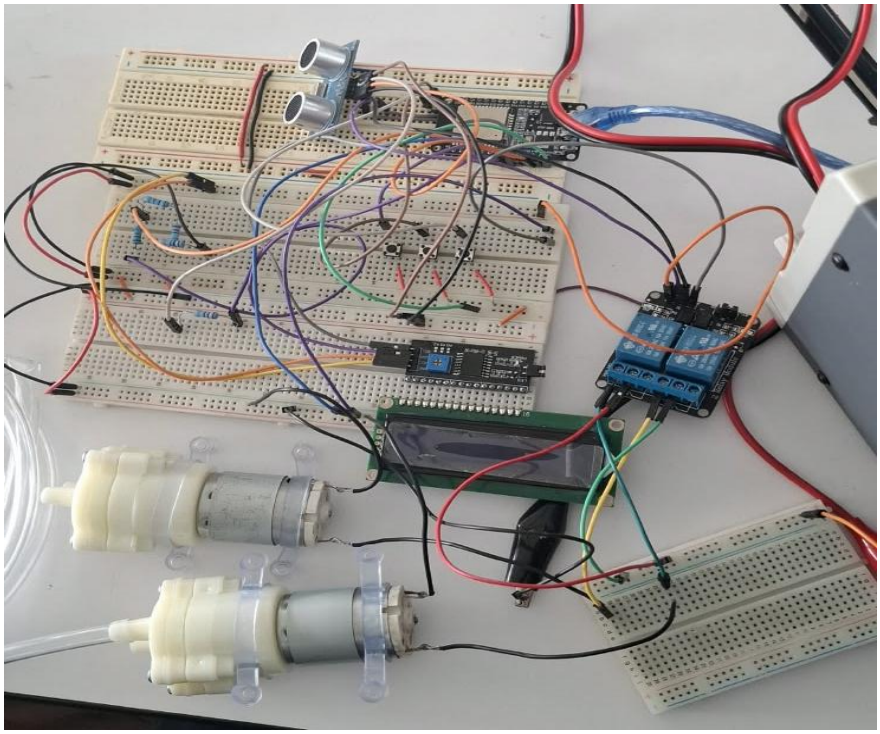


Figura 3. Circuito físico con bombas sin activar

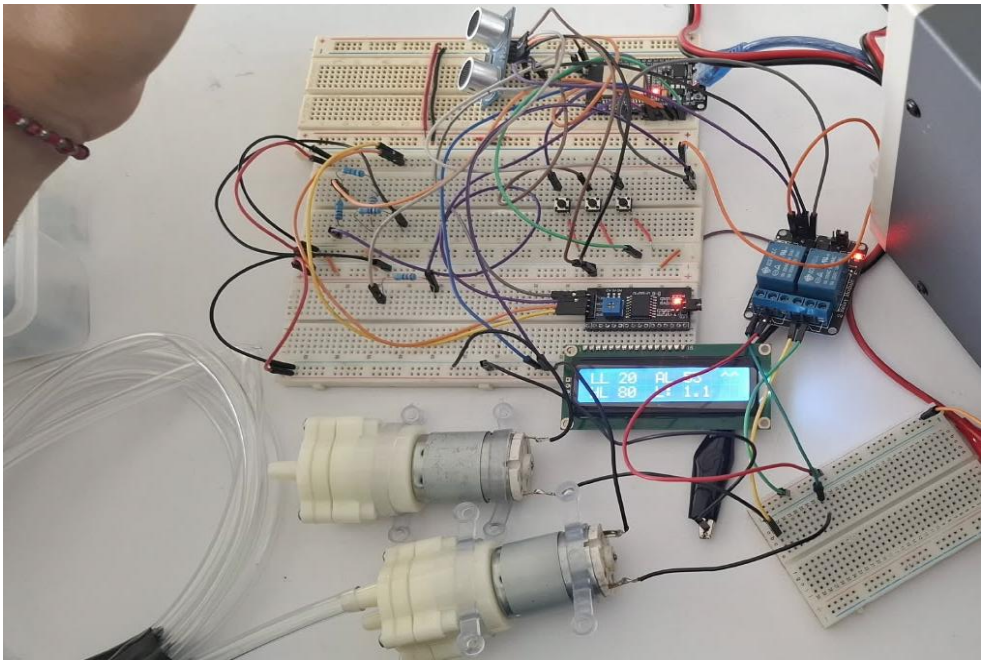


Figura 4. Circuito físico con bombas en funcionamiento.

Versión 3 – Código Final

```
#include "Wire.h"
#include "LiquidCrystal_I2C.h"

// ==================== PINES ====================
const int TRIG_PIN = 13;
const int ECHO_PIN = 14;
const int BTN_SELECT = 18;
const int BTN_UP     = 32;
const int BTN_DOWN   = 19;
const int RELAY_LLENAR = 4;
const int RELAY_DRENAR = 5;

// ==================== CONFIGURACIÓN RELÉ ====================
const int RELAY_ON  = LOW;
const int RELAY_OFF = HIGH;

// ==================== PARÁMETROS DEL TANQUE ====================
const float DISTANCIA_VACIO = 11.5;
const float DISTANCIA_LLENO = 4.0;
const float ALTURA_MAXIMA   = DISTANCIA_VACIO - DISTANCIA_LLENO;
const float CAPACIDAD_LITROS = 0.589;

// ==================== AJUSTES ====================
const int PASO = 5;

// ==================== VARIABLES ====================
int LL = 20;
int HL = 80;
int AL = 0;
float litrosActuales = 0.0;

enum Estado { ESPERA, LLENANDO, DRENANDO };
Estado estadoActual = ESPERA;

unsigned long ultimaLecturaBotones = 0;
unsigned long ultimaMedicion = 0;
unsigned long ultimoChequeoI2C = 0;

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ==================== SETUP ====================
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("=== Modo FUENTE ===");
  Serial.println("Sube LL y HL para empezar el ciclo");

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BTN_SELECT, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(RELAY_LLENAR, OUTPUT);
  pinMode(RELAY_DRENAR, OUTPUT);
  digitalWrite(RELAY_LLENAR, RELAY_OFF);
  digitalWrite(RELAY_DRENAR, RELAY_OFF);

  Wire.begin(26, 27);
  Wire.setClock(100000);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Iniciando...");
  delay(1000);
}
```

```
// ==================== RECUPERACIÓN I2C ====================
void recoverI2C() {
  Serial.println("Reiniciando I2C...");
  Wire.end();
  pinMode(26, INPUT_PULLUP);
  pinMode(27, INPUT_PULLUP);
  delay(10);

  if (digitalRead(26) == LOW) {
    pinMode(27, OUTPUT);
    for (int i = 0; i < 16; i++) {
      digitalWrite(27, HIGH); delayMicroseconds(5);
      digitalWrite(27, LOW);  delayMicroseconds(5);
    }
    pinMode(26, OUTPUT);
    digitalWrite(26, HIGH); delayMicroseconds(5);
    digitalWrite(27, HIGH); delayMicroseconds(5);
  }

  Wire.begin(26, 27);
  Wire.setClock(100000);
  lcd.init();
  lcd.backlight();
  Serial.println("I2C recuperado");
}

// ==================== MEDIR DISTANCIA ====================
float medirDistancia() {
  float medidas[5];
  int validas = 0;

  for (int i = 0; i < 5; i++) {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duracion = pulseIn(ECHO_PIN, HIGH, 30000);
    float d = duracion * 0.0343 / 2;

    if (d > 0.5 && d < 30) {
      medidas[validas] = d;
      validas++;
    }
    delay(3);
  }

  if (validas == 0) return DISTANCIA_VACIO;

  for (int i = 0; i < validas - 1; i++) {
    for (int j = 0; j < validas - i - 1; j++) {
      if (medidas[j] > medidas[j + 1]) {
        float temp = medidas[j];
        medidas[j] = medidas[j + 1];
        medidas[j + 1] = temp;
      }
    }
  }

  return medidas[validas / 2];
}

// ==================== CONTROL MODO FUENTE ====================
void controlarFuente() {
  // Si no hay valores definidos, no hacer nada
  if (HL == 0 || LL == 0 || HL <= LL) {
    digitalWrite(RELAY_LLENAR, RELAY_OFF);
```

```
    digitalWrite(RELAY_DRENAR, RELAY_OFF);
    estadoActual = ESPERA;
    return;
  }

  // ===== ESTADO: LLENANDO =====
  if (estadoActual == LLENANDO) {
    digitalWrite(RELAY_DRENAR, RELAY_OFF);
    digitalWrite(RELAY_LLENAR, RELAY_ON);

    // Si llegó a HL → cambiar a DRENANDO
    if (AL >= HL) {
      estadoActual = DRENANDO;
      digitalWrite(RELAY_LLENAR, RELAY_OFF);
      digitalWrite(RELAY_DRENAR, RELAY_ON);
      Serial.println("HL alcanzado → DRENANDO");
    }
  }

  // ===== ESTADO: DRENANDO =====
  else if (estadoActual == DRENANDO) {
    digitalWrite(RELAY_LLENAR, RELAY_OFF);
    digitalWrite(RELAY_DRENAR, RELAY_ON);

    // Si llegó a LL → cambiar a LLENANDO
    if (AL <= LL) {
      estadoActual = LLENANDO;
      digitalWrite(RELAY_DRENAR, RELAY_OFF);
      digitalWrite(RELAY_LLENAR, RELAY_ON);
      Serial.println("LL alcanzado → LLENANDO");
    }
  }

  // ===== ESTADO: ESPERA (inicio) =====
  else {
    // Si AL <= LL → empezar a llenar
    if (AL <= LL) {
      estadoActual = LLENANDO;
      digitalWrite(RELAY_LLENAR, RELAY_ON);
      digitalWrite(RELAY_DRENAR, RELAY_OFF);
      Serial.println("Iniciando ciclo → LLENANDO");
    }
    // Si AL >= HL → empezar a drenar
    else if (AL >= HL) {
      estadoActual = DRENANDO;
      digitalWrite(RELAY_LLENAR, RELAY_OFF);
      digitalWrite(RELAY_DRENAR, RELAY_ON);
      Serial.println("Iniciando ciclo → DRENANDO");
    }
  }
}

// ==================== LOOP ====================
void loop() {
  unsigned long ahora = millis();

  // ===== LEER BOTONES CADA 150ms =====
  if (ahora - ultimaLecturaBotones >= 150) {
    ultimaLecturaBotones = ahora;

    bool select = (digitalRead(BTN_SELECT) == LOW);
    bool up     = (digitalRead(BTN_UP) == LOW);
    bool down   = (digitalRead(BTN_DOWN) == LOW);

    if (select) {
      if (up)   { HL += PASO; if (HL > 100) HL = 100; Serial.print("HL = ");
Serial.println(HL); }
      if (down) { HL -= PASO; if (HL < 0) HL = 0;     Serial.print("HL = ");
Serial.println(HL); }
```

```
    } else {
      if (up)   { LL += PASO; if (LL > 100) LL = 100; Serial.print("LL = ");
Serial.println(LL); }
      if (down) { LL -= PASO; if (LL < 0) LL = 0;     Serial.print("LL = ");
Serial.println(LL); }
    }

    // Si HL <= LL, ajustar
    if (HL <= LL && HL > 0 && LL > 0) {
      if (select) {
        LL = HL - PASO;
        if (LL < 0) LL = 0;
      }
    }
  }

  // ===== MEDIR SENSOR CADA 100ms =====
  if (ahora - ultimaMedicion >= 100) {
    ultimaMedicion = ahora;

    float distancia = medirDistancia();
    if (distancia < DISTANCIA_LLENO || distancia <= 0) distancia =
DISTANCIA_LLENO;
    if (distancia > DISTANCIA_VACIO) distancia = DISTANCIA_VACIO;

    float nivelAguaCm = DISTANCIA_VACIO - distancia;
    if (nivelAguaCm < 0) nivelAguaCm = 0;
    if (nivelAguaCm > ALTURA_MAXIMA) nivelAguaCm = ALTURA_MAXIMA;

    AL = (int)((nivelAguaCm / ALTURA_MAXIMA) * 100.0);
    litrosActuales = (AL / 100.0) * CAPACIDAD_LITROS;
  }

  // ===== CONTROL MODO FUENTE =====
  controlarFuente();

  // ===== PANTALLA =====
  char fila0[17];
  char fila1[17];
  const char* indicador = "  ";
  if (estadoActual == LLENANDO)      indicador = "^^";
  else if (estadoActual == DRENANDO) indicador = "vv";
  else                               indicador = "--";

  snprintf(fila0, sizeof(fila0), "LL %-3d AL %-3d %s", LL, AL, indicador);
  snprintf(fila1, sizeof(fila1), "HL %-3d L: %-4.2f  ", HL, litrosActuales);

  lcd.setCursor(0, 0); lcd.print(fila0);
  lcd.setCursor(0, 1); lcd.print(fila1);

  // ===== CHEQUEO I2C =====
  if (ahora - ultimoChequeoI2C > 2000) {
    ultimoChequeoI2C = ahora;
    Wire.beginTransmission(0x27);
    if (Wire.endTransmission() != 0) {
      Serial.println("⚠ LCD no responde, reiniciando I2C...");
      recoverI2C();
    }
  }

  delay(10);
}
```

- `medirDistancia()` toma 5 lecturas, descarta las inválidas, las ordena y devuelve la mediana.

- `controlarFuente()` implementa la máquina de estados ESPERA, LLENANDO y DRENANDO.
- `recoverI2C()` reinicia el bus y la LCD si deja de responder.
- El `loop()` usa `millis()` para botones (150 ms), sensor (100 ms) y chequeo I2C (2 s).

Funcionamiento

1. Al iniciar, se abre la comunicación serial a 115200 baudios, se configuran los pines y ambos relevadores quedan apagados (HIGH, por ser activos en bajo). Se inicializa el bus I2C a 100 kHz y la LCD muestra “Iniciando...” durante 1 segundo.
2. Cada 150 ms se leen los botones. Sin SELECT, UP y DOWN modifican LL en pasos de 5 %; con SELECT presionado, modifican HL. Ambos se limitan entre 0 y 100 %.
3. Cada 100 ms se mide el nivel: se toman cinco lecturas del HC-SR04, se descartan las inválidas y se usa la mediana. Con ella se calcula el nivel (11.5 cm menos la distancia medida), el porcentaje de llenado (AL) y los litros (AL/100 × 0.589 L).
4. `controlarFuente()` compara AL con los umbrales. Si AL ≤ LL, activa el relevador de llenado. Al llegar a AL ≥ HL, apaga el llenado y activa el drenado. Al llegar a AL ≤ LL, vuelve a llenar, y el ciclo se repite.
5. Si HL ≤ LL o alguno vale 0, ambos relevadores se apagan. Si el nivel inicial está entre LL y HL, el sistema espera hasta que los umbrales se ajusten de modo que el nivel quede fuera de ellos.
6. La LCD se actualiza continuamente: la fila 1 muestra LL, AL y un indicador de estado (^^ llenando, vv drenando, -- en espera), y la fila 2 muestra HL y los litros.
7. Cada 2 segundos se verifica que la LCD responda; si no lo hace, se reinicia el bus I2C.

Resultados

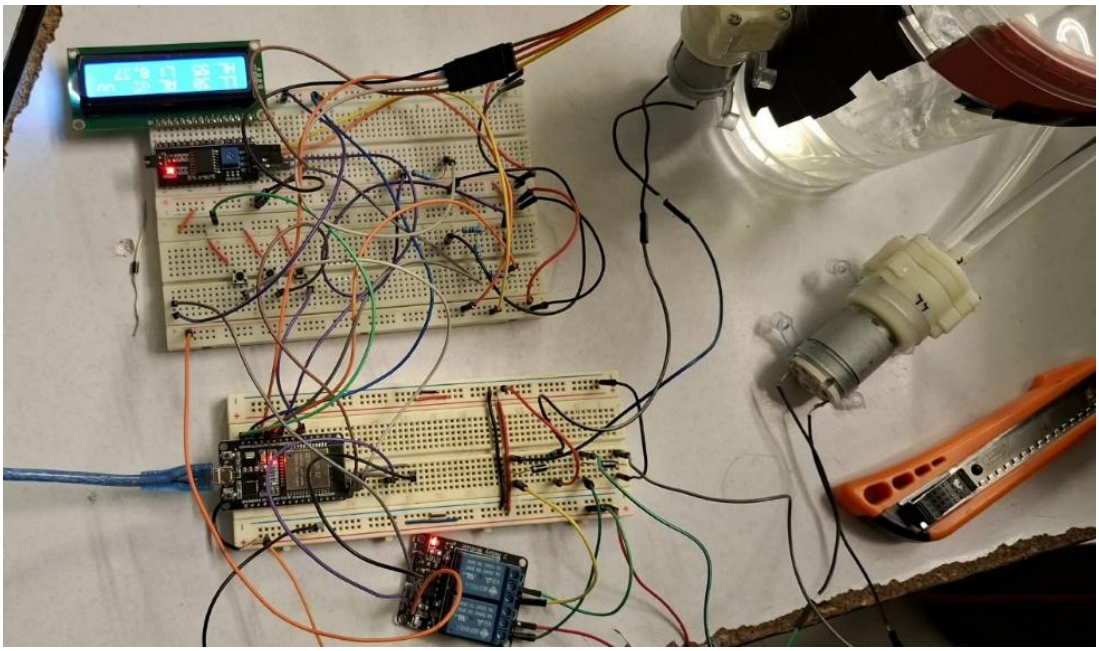


Figura 5. Circuito físico funcionando

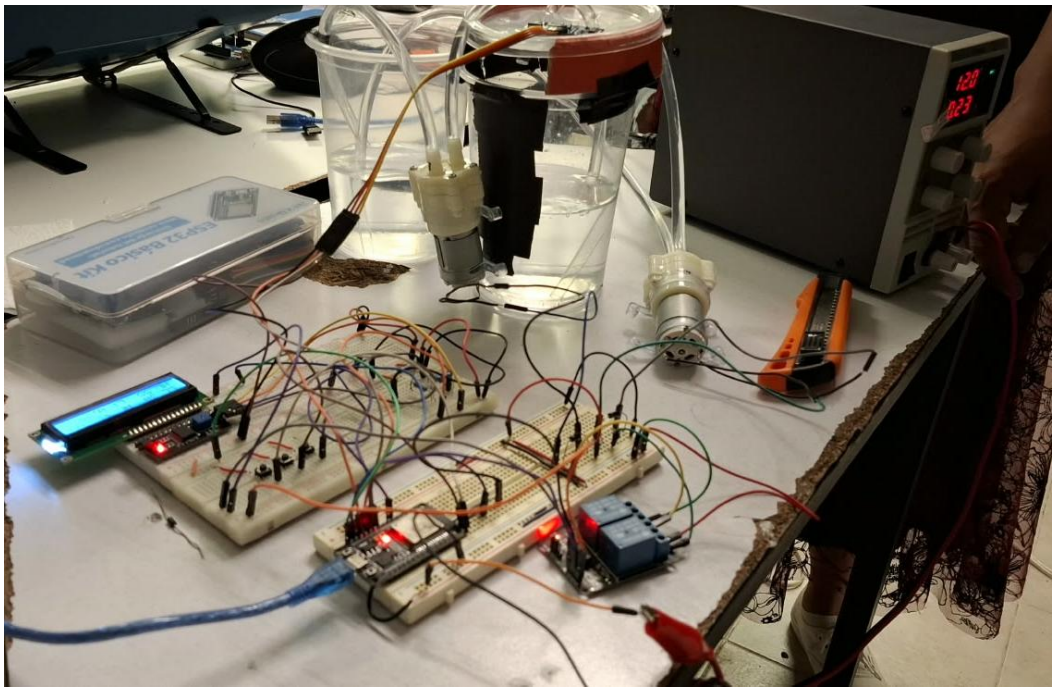


Figura 6. Circuito físico funcionando vista completa

Durante las pruebas, la primera versión (sin bombas) confirmó que el sensor medía correctamente la distancia y que los botones ajustaban LL y HL, mostrándose los valores en la LCD.

Al incorporar los relevadores se presentaron tres fallos:

- **Conexión incorrecta del relevador:** dañó (quemó) el módulo de conexiones del ESP32.
- **Doble activación de botones:** como consecuencia del mismo fallo eléctrico, una sola pulsación se interpretaba como dos, alterando LL y HL de forma errática.
- **Parpadeo en la LCD:** por un mal contacto en el adaptador I2C, afectando la legibilidad.

Tras corregir la conexión de los relevadores (verificando su pinout antes de energizar), asegurar el contacto del adaptador I2C y adoptar la versión final del programa, el sistema funcionó de manera estable. Con el nivel en LL o por debajo, la bomba de llenado se activó hasta alcanzar HL; después se activó la de drenado hasta volver a LL, repitiéndose el ciclo.

Observaciones y conclusiones

Esta práctica representó un salto de complejidad respecto a las anteriores, al combinar un sensor de distancia, una interfaz con varios botones, comunicación I2C, una máquina de estados y el control de cargas de potencia mediante relevadores. El desarrollo en tres etapas (monitoreo, control con bombas y versión final) permitió validar cada parte antes de agregar la siguiente y corregir los problemas con una base funcional.

Se aprendió que una conexión incorrecta en el circuito de potencia puede dañar físicamente la tarjeta, como ocurrió con el módulo de conexiones del ESP32. Esto muestra la importancia de revisar el pinout del módulo relevador (VCC, GND, IN, COM/NO/NC) antes de energizar y de alimentar las bombas con una fuente separada. También se comprobó que los módulos pueden ser activos en alto o en bajo; en la versión final bastó definir `RELAY_ON = LOW` y `RELAY_OFF = HIGH`, y el resto del código no cambió. La doble activación de los botones mostró cómo una falla en una parte del circuito puede afectar entradas que parecen no relacionadas, lo que refuerza la importancia de una buena tierra común y de separar la sección de control de la de potencia. En el software, la lectura de botones espaciada cada 150 ms con `millis()` ayuda a que una pulsación no se interprete varias veces, pero la causa de fondo era eléctrica.

El parpadeo de la LCD confirmó que muchas fallas en sistemas embebidos son físicas, no del programa. Aun así, la rutina de recuperación del I2C añadió una protección por software ante pérdidas de comunicación. Por su parte, el filtro de mediana de cinco lecturas permite descartar mediciones erráticas del sensor ultrasónico, que pueden variar por reflexiones o por el movimiento del líquido.

En cuanto a la relación entre programa y circuito, el software debe diseñarse junto con el hardware: pines I2C personalizados, relevadores activos en bajo, dimensiones reales del tanque (11.5 cm y 0.589 L) y el comportamiento de las bombas. Se comprobó que un control ON-OFF de nivel con dos bombas es viable con componentes de bajo costo. Como mejora, podría reincorporarse la memoria no volátil de la Versión 2 para conservar LL y HL tras un reinicio.

Referencias

[1] TRBL Services. (s. f.). *Sistema embebido y sus características: Conceptos fundamentales*. TRBL Services. https://trbl-services.eu/blog-sistema-embebido-caracteristicas/

[2] SD Industrial. (s. f.). *Sistemas embebidos*. SD Industrial. https://sdindustrial.com.mx/blog/sistemas-embebidos/

[3] Llamas, L. (s. f.). *Pinout y detalles del hardware del ESP32*. Luis Llamas. https://www.luisllamas.es/esp32-detalles-hardware-pinout/

[4] Acebott. (s. f.). *Introducción al ESP32*. Acebott. https://acebott.com/es/docs/getting-started-with-esp32/

[5] Panama Hitek. (s. f.). *Entradas y salidas analógicas y digitales*. Panama Hitek. https://panamahitek.com/entradas-y-salidas-analogicas-y-digitales/

[6] MCI Electronics. (2022). *Niveles lógicos*. MCI Electronics. https://cursos.mcielectronics.cl/2022/08/05/niveles-logicos/

[7] All About Circuits. (s. f.). *Niveles de voltaje de señales lógicas*. All About Circuits. https://www.allaboutcircuits.com/textbook/digital/chpt-3/logic-signal-voltage-levels/

[8] Datatechnic. (s. f.). *Control eléctrico: Fundamentos y aplicaciones en la industria moderna*. Datatechnic. https://datatechnic.com.mx/control-electrico-conceptos-basicos-y-aplicaciones-en-la-industria/

[9] OMRON. (s. f.). *What is "hysteresis"?* OMRON Industrial Automation. https://www.ia.omron.com/support/faq/answer/53/faq00549/

[10] Llamas, L. (s. f.). *On/Off control with hysteresis*. Luis Llamas. https://www.luisllamas.es/en/on-off-control-with-hysteresis/

[11] SunFounder. (s. f.). *Módulo ultrasónico (HC-SR04)*. SunFounder. https://github.com/sunfounder/esp32-starter-kit/blob/docs-es/docs/source/components/component_ultrasonic_module.rst

[12] Microcontrollers Lab. (s. f.). *HC-SR04 ultrasonic sensor with ESP32 – Measure distance*. Microcontrollers Lab. https://microcontrollerslab.com/hc-sr04-ultrasonic-esp32-tutorial/

[13] Random Nerd Tutorials. (s. f.). *ESP32 I2C communication: Set pins, multiple bus interfaces and peripherals (Arduino IDE)*. Random Nerd Tutorials. https://randomnerdtutorials.com/?p=89386

[14] Last Minute Engineers. (s. f.). *How to use an I2C LCD display with ESP32*. Last Minute Engineers. https://lastminuteengineers.com/esp32-i2c-lcd-tutorial/

[15] Keyestudio. (2024). *How to use relay with ESP32*. Keyestudio. https://keyestudio.com/blog/how-to-use-relay-with-esp32-241

[16] Unit Electronics. (s. f.). *LA-521 Bomba de diafragma 12V DC*. Unit Electronics. https://uelectronics.com/producto/la-521-bomba-de-diafragma-12v-dc/